User Stories and Acceptance Criteria: E-Commerce API

Step 6 of the Postman BA mini project | Source: Postman test cases TC-01 to TC-12 (DummyJSON)

Purpose

These user stories and acceptance criteria turn the API behavior observed in Postman into requirements. Every criterion is written in Given/When/Then form and linked to the test case that supports it. Each criterion also carries an **evidence status** so it is clear what was actually observed and what is still an assumption.

Evidence status legend

Status	Meaning
Verified	Observed directly in the Postman results (status code, message, fields)
To verify	Reasonable expectation that has not been tested yet; check it in Postman
Pending re-run	Depends on TC-12, which must be re-run as POST /products/add with body {}
Assumption	Proposed by the BA (e.g. a performance target); needs business agreement
Gap observed	The API behavior differs from what a real shop would need

Story summary

ID	User story	Priority	Linked test cases
US-01	Log in with username and password	Must	TC-01, TC-08
US-02	View my profile (protected)	Must	TC-02, TC-09, TC-10
US-03	Browse the product list	Must	TC-03
US-04	View one product's details	Must	TC-04, TC-11
US-05	Add a product (admin)	Should	TC-05, TC-12
US-06	Update a product (admin)	Should	TC-06
US-07	Delete a product (admin)	Should	TC-07, TC-11
US-08	Restrict product management to admins	Must	TC-05, TC-06, TC-07

US-01: Log in

As a registered customer, **I want to** log in with my username and password, **so that** I can access my account.

Priority: Must have | **Linked test cases:** TC-01, TC-08

1. **Given** a registered user with valid credentials, **when** they submit the login request, **then** the system returns 200 with an access token, a refresh token, the user ID and the username. *(TC-01, Verified)*
2. **Given** a registered user who enters an incorrect password, **when** they submit the login request, **then** the system returns 400 with the message "Invalid Credentials" and does not return a token. *(TC-08, Verified)*

US-02: View my profile

As a logged-in customer, **I want to** view my profile details, **so that** I can confirm my account information is correct.

Priority: Must have | **Linked test cases:** TC-02, TC-09, TC-10

1. **Given** a logged-in user with a valid access token, **when** they request their profile, **then** the system returns 200 with their details, including ID, username, email, first name and last name. *(TC-02, Verified)*
2. **Given** a request with no access token, **when** the user requests their profile, **then** the system returns 401 with the message "Access Token Is Required". *(TC-09, Verified)*
3. **Given** a request with an invalid or expired access token, **when** the user requests their profile, **then** the system returns 401 with the message "Invalid/Expired Token". *(TC-10, Verified)*

US-03: Browse the product list

As a customer, **I want to** browse the list of available products, **so that** I can find something to buy.

Priority: Must have | **Linked test cases:** TC-03

1. **Given** a customer browsing the catalog (no login required), **when** they request the product list, **then** the system returns 200 with a list of products and the total, skip and limit values. *(TC-03, Verified)*
2. **Given** a product list request, **when** the customer supplies limit and skip values, **then** the system returns only the requested number of products, starting from the requested position. *(TC-03, To verify: Test the limit and skip query parameters in Postman)*
3. **Given** a normal product list request, **when** it is submitted, **then** the system responds within 2,000 ms. *(TC-03, Assumption: Proposed target to be agreed with the business; the single run in TC-03 took 2,900 ms)*

US-04: View one product's details

As a customer, **I want to** view the details of a single product, **so that** I can decide whether to buy it.

Priority: Must have | **Linked test cases:** TC-04, TC-11

1. **Given** a product ID that exists, **when** the customer requests that product, **then** the system returns 200 with the product's ID, title, price, category and stock. *(TC-04, Verified)*
2. **Given** a product ID that does not exist, **when** the customer requests that product, **then** the system returns 404 with a message stating that the product with that ID was not found. *(TC-11, Verified)*

US-05: Add a product

As a store admin, **I want to** add a new product to the catalog, **so that** customers can see and buy it.

Priority: Should have | **Linked test cases:** TC-05, TC-12

1. **Given** an admin who supplies a title and a price, **when** they submit the add-product request, **then** the system returns 201 with the new product's ID, title and price. *(TC-05, Verified)*
2. **Given** an admin who submits the add-product request with an empty body, **when** the request is processed, **then** the system rejects it with 400 and a message that names the missing required fields. *(TC-12, Pending re-run: 400 is a BA assumption; re-run TC-12 as POST /products/add with body {})*
3. **Given** a product was added successfully, **when** the admin requests it using the returned ID, **then** the system returns that product. *(TC-05, To verify: Call GET on the returned ID after the add)*

US-06: Update a product

As a store admin, **I want to** update an existing product's details, **so that** the catalog stays accurate.

Priority: Should have | **Linked test cases:** TC-06

1. **Given** an existing product ID and a changed field (for example the title), **when** the admin submits the update request, **then** the system returns 200 with the updated values. *(TC-06, Verified)*
2. **Given** a product was updated successfully, **when** the product is requested again, **then** the system returns the updated values. *(TC-06, To verify: Call GET on the same ID after the update)*

US-07: Delete a product

As a store admin, **I want to** delete a product that is no longer sold, **so that** customers do not see it.

Priority: Should have | **Linked test cases:** TC-07, TC-11

1. **Given** an existing product ID, **when** the admin submits the delete request, **then** the system returns 200 with the deleted product's data. *(TC-07, Verified)*
2. **Given** a product was deleted successfully, **when** it is requested by ID, **then** the system returns 404. *(TC-07, TC-11, To verify: Call GET on the same ID after the delete)*

US-08: Restrict product management to admins

As a store owner, **I want** adding, updating and deleting products to be limited to authorized admins, **so that** customers cannot change the catalog.

Priority: Must have | **Linked test cases:** TC-05, TC-06, TC-07

1. **Given** a request to add, update or delete a product with no valid access token, **when** it is submitted, **then** the system denies it with 401. *(TC-05, TC-06, TC-07, Gap observed: All three calls succeeded with no token (Auth Needed? = No))*
2. **Given** a logged-in user who does not have admin rights, **when** they attempt to add, update or delete a product, **then** the system denies it with 403. *(TC-05, TC-06, TC-07, Assumption: Needs a defined role model; not tested)*

Assumptions and notes

- The 2,000 ms response-time target in US-03 is a proposal, not an observed requirement. Replace it with a target agreed with the business.
- The 400 status for an empty body in US-05 is a BA assumption. Update it after TC-12 is re-run.
- Criteria marked To verify should be checked in Postman and their status changed to Verified, or rewritten if the API behaves differently.
- US-08 comes directly from the "Auth Needed?" column of the test results and feeds the gap analysis (Step 7).

Next steps

1. Re-run TC-12 as POST {{base_url}}/products/add with body {} and update AC 5.2.
2. Check the To verify criteria in Postman (pagination, persistence after add, update and delete).
3. Move on to Step 7: gap analysis, using the Gap observed and Assumption items above.